SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninfor
solve problems effectively. *(BL3)*

Assessment Tool Weightage: 30%

Duration : 45 Minutes

QUESTIONS: 2 **Total Marks:20**

Annexure A

Descriptive Written Test

Code of Conduct:

I Sharwin 192521207 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Sharwin

Annexure A

Descriptive Written Test

1. A smart home automation system is being designed to manage lighting, temperature, and security based on user behavior and environmental conditions. The system must respond to real-time inputs such as motion detection, temperature changes, and time of day while also learning user preferences over time. In this context, explain the different types of intelligent agents (simple reflex, model-based, goal-based, and utility-based agents) and analyze how each type would function within this system. Justify which type of agent or hybrid approach would be most effective for ensuring comfort, energy efficiency, and security.

2. A robot is placed in an unknown maze and must find a path to the exit without prior knowledge of the layout. Explain how uninformed search strategies like Uniform Cost Search (UCS) and Iterative Deepening Search (IDS) can help solve this problem. Discuss the advantages and disadvantages of these algorithms in terms of time and space complexity. Relate the problem to different types of intelligent agents and explain how agent design affects decision-making. Suggest which combination of agent type and search strategy would be most effective and justify your choice.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Criteria	**Level 4 (Exemplary)**	**Level 3 (Proficient)**	**Level 2 (Developing)**	**Level 1 (Beginning)**	**Max Marks**
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, wellexplained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Wellstructured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

1. Smart Home Automation System and Types of Intelligent Agents

Introduction

A **smart home automation system** is an intelligent system that controls different devices in a house, such as:

- Lights
- Fans
- Air conditioners
- Heating systems
- Security cameras
- Door locks
- Alarms

The system receives information from sensors such as:

- Motion sensors
- Temperature sensors
- Light sensors
- Door/window sensors
- Smoke sensors
- Time/date information

The main objective is to provide **comfort, energy efficiency, convenience, and security**. An intelligent agent can be defined as a system that **perceives its environment through sensors and performs actions through actuators**.

Smart Home Environment
↓
Sensors
↓
Percepts
↓
Intelligent Agent
↓
Decisions
↓
Actuators
↓
Lights / AC / Locks / Alarm

A. Simple Reflex Agent

A **Simple Reflex Agent** selects an action based only on the **current percept**.
It follows condition-action rules:
IF condition THEN action
It does not maintain a detailed memory of previous situations.

Examples in Smart Home

Lighting
IF motion detected
THEN turn ON the light

Temperature
IF temperature > 28°C
THEN turn ON the AC

Security
IF door opens while security mode is ON
THEN activate alarm

Advantages

- Simple to design
- Fast response
- Easy to implement
- Requires less computational power
- Suitable for basic automation

Disadvantages

- Does not remember previous situations
- Cannot understand the complete environment
- Cannot learn user preferences
- May make incorrect decisions in complex situations

Example
Suppose the user usually turns the lights off after 11 PM.
A simple reflex agent does not remember this preference. If motion is detected at 11:30 PM, it may simply turn the light ON.

B. Model-Based Agent
A **Model-Based Agent** maintains an **internal model or internal state** of the environment. It can remember information about previous situations.

Example
The agent can remember:

- Whether the user is at home
- Which rooms are occupied
- Previous temperature
- Door status
- Security status
- Recent sensor readings For example:

Motion detected
↓
Check internal state
↓
Is person present?

↓

Yes → Turn ON light

No → Keep light OFF

Smart Home Example

Suppose a person enters the living room.

The model-based agent can consider:

- Current motion
- Previous motion
- Time of day
- Whether the person is already in the house
- Current lighting condition

It can then decide whether the light should be turned ON.

Advantages

- Maintains internal state
- Handles partially observable environments
- Makes better decisions than simple reflex agents
- Can consider previous information

Disadvantages

- More complex
- Requires memory
- Requires an accurate model of the environment
- More computational resources may be needed

C. Goal-Based Agent

A **Goal-Based Agent** selects actions based on a desired goal. Instead of simply reacting to a sensor, it asks:

"What action will help me achieve my goal?"

Smart Home Goals Examples:

- Maintain comfortable temperature
- Save energy
- Keep the house secure
- Turn off unnecessary appliances
- Prepare the house before the user arrives

Example Goal:

Maintain room temperature around 24°C.

The agent observes:

Current temperature = 30°C

Goal temperature = 24°C It decides:

Turn ON AC

↓

Monitor temperature

↓

Temperature reaches 24°C

↓

Reduce/turn OFF cooling
Security Example Goal:
Maintain a secure house.
The agent can:

1. Lock doors.
2. Monitor windows.
3. Detect unusual motion.
4. Activate an alarm if required.

Advantages

- Works toward specific objectives
- Can plan actions
- More flexible than a simple reflex agent
- Suitable for complex tasks

Disadvantages

- Goal achievement alone may not consider the cost of an action
- May select a technically successful but inefficient action
- Requires goal representation and planning

D. Utility-Based Agent
A **Utility-Based Agent** considers how desirable each possible outcome is.
It does not simply ask:
"Will this action achieve the goal?" It
also asks:
"Which available action provides the most desirable result?" The
utility can consider:

- Comfort
- Energy consumption
- Security
- Cost
- User preferences
- Convenience

Example
Suppose the room temperature is 27°C.
The agent has two choices:
Option 1: Turn AC ON at high power.
Option 2: Turn fan ON and use moderate cooling.
The agent can compare:

Factor	**Option 1**	**Option 2**
Comfort	High	Medium/High
Energy consumption	High	Lower
Cost	High	Lower
Goal achievement	Yes	Yes

The utility-based agent can choose the action that provides a better overall balance.

Comparison of Four Agents

Feature	Simple Reflex	Model-Based	Goal-Based	Utility-Based
Current input	Yes	Yes	Yes	Yes
Memory/internal state	No	Yes	Yes	Yes
Goals	No	No/limited	Yes	Yes
Utility	No	No	No	Yes
Learning	No	Limited	Limited	Can support
Complexity	Low	Medium	Medium/High	High
Smart-home suitability	Basic	Good	Very Good	Excellent for trade-offs

How Each Agent Functions in Smart Home

Simple Reflex

Motion detected
↓
Turn ON light

Useful for simple immediate responses.

Model-Based

Motion + Previous State + Time
↓
Internal Model
↓
Decide Action

Useful when the system needs memory.

Goal-Based

Goal: Comfortable Room
↓
Check Temperature
↓
Select Action
↓
Reach Desired Temperature

Useful for achieving specific objectives.

Utility-Based

Comfort
+
Energy Efficiency
+
Security
+
User Preference

↓
Utility Calculation
↓
Best Available Action
Useful when several objectives must be balanced.

Most Effective Approach
For a modern smart-home system, a **hybrid approach combining model-based, goal-based, utility-based, and learning components** would be appropriate.
A possible architecture is:

Sensors
↓
Current Environment
↓
Internal Model
↓
User Preferences
↓
Goals
↓
Utility Evaluation
↓
Decision Making
↓
Actions
↓
Lights / AC / Security / Appliances
↓
Feedback
↓
Learning

Why Hybrid?
A smart home has multiple requirements.
Comfort
The system should maintain suitable:

- Temperature
- Lighting
- Room conditions

Energy Efficiency
The system should avoid unnecessary:

- Lighting
- AC usage
- Heating
- Appliance operation

Security
The system should monitor:

- Doors
- Windows
- Motion
- Cameras
- Alarms

Personalization

The system should learn preferences such as:

- Preferred temperature
- Preferred lighting
- Typical arrival time
- Sleeping schedule
- Frequently used rooms

A hybrid agent can use **simple reflex rules for immediate safety responses**, a **model-based component for maintaining state**, **goal-based planning for achieving desired conditions**, and **utility-based decision-making for balancing comfort, energy, and security**.

Example Scenario

Suppose it is 10:00 PM.

The system detects:

- No motion in the living room
- User is sleeping
- Bedroom temperature = 24°C
- Living-room lights are ON
- Front door is unlocked A hybrid agent can reason:

1. No living-room activity → turn off unnecessary lights.
2. User is sleeping → maintain bedroom temperature.
3. Front door unlocked → lock the door.
4. Security mode → activate appropriate monitoring.
5. Avoid unnecessary AC operation → save energy.

This is more effective than simply reacting to one sensor.

Conclusion

Different intelligent-agent types provide different levels of decision-making. **Simple reflex agents** are suitable for basic immediate actions, while **model-based agents** can maintain information about the environment. **Goal-based agents** work toward desired objectives, and **utility-based agents** can balance multiple competing requirements.

For a smart home that must provide **comfort, energy efficiency, security, and personalization**, a **hybrid model-based, goal-based and utility-based architecture with learning capabilities** is suitable.

---

2. Robot in an Unknown Maze – UCS and IDS Introduction

Consider a robot placed in an unknown maze.

The robot:

- Starts at a starting position
- Does not initially know the complete maze
- Must find an exit
- Can move through available paths
- Must avoid blocked paths

This is a classical **AI search problem**.
The robot can use uninformed search algorithms such as:

- **Uniform Cost Search (UCS)**
- **Iterative Deepening Search (IDS)**

These algorithms do not require a heuristic estimate of the distance to the goal.

Problem Formulation
Initial State
The robot's starting location.
$$\boxed{S}$$
Goal State The
maze exit.
$$\boxed{G}$$
Actions
Depending on the maze, the robot may:

- Move Up
- Move Down
- Move Left
- Move Right

Transition Model
An action changes the robot's current position.
For example:
Current position
↓
Move Right
↓
New position
Path Cost
If every movement costs 1:
$$g(n)=\text{number of movements}$$
The objective is to find a path from S to G.

A. Uniform Cost Search
Definition
Uniform Cost Search expands the node having the **lowest cumulative path cost**.
It uses:
$$\boxed{f(n)=g(n)}$$ where:
$$g(n)=\text{actual cost from Start to node n}$$
If every movement has cost 1, UCS behaves similarly to BFS in terms of which depth is expanded first.

UCS Example
Consider:



Assume:
S → A = 1
S → B = 1
A → C = 1
B → D = 1
C → G = 1 D → G = 1
Start: \[
g(S)=0 \]
Expand S:
A = 1
B = 1
Priority queue:
A(1), B(1) Expand
A:
C = 2 Queue:
B(1), C(2)
Expand B: D
= 2 Queue:
C(2), D(2)
Expand C: G
= 3 Goal
reached.
Therefore:
\[\boxed{S\rightarrow A\rightarrow C\rightarrow G} \] Cost:
\[\boxed{3} \]

UCS Advantages
1. Optimal
If all step costs are non-negative, UCS finds a least-cost solution. **2.**
Complete
It can find a solution if one exists under the usual UCS assumptions, particularly positive step costs.
3. Handles Different Costs Suppose:
Path A = 5
Path B = 10
UCS considers the actual cost and chooses the lower-cost route.

UCS Disadvantages

1. Memory Consumption

UCS stores many frontier nodes.

Therefore, it can require large amounts of memory.

2. Slow for Large Search Spaces

If many paths have similar low costs, UCS may expand many nodes before reaching the goal.

3. No Goal Direction

UCS does not know which direction is closer to the goal because it does not use a heuristic.

UCS Complexity

Let:

- \(b\) = branching factor
- \(C^*\) = cost of optimal solution
- \(\epsilon\) = minimum positive step cost The approximate time and space complexity is: \[O\left(b^{1+\lfloor C^*/\epsilon\rfloor}\right) \] Both time and space can become large.

For equal step costs, UCS has behavior similar to BFS.

B. Iterative Deepening Search

Definition

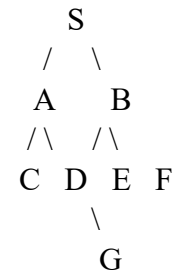
Iterative Deepening Search (IDS) combines important properties of:

- Depth First Search
- Breadth First Search

It repeatedly performs depth-limited DFS with increasing depth limits. The depth limits are:

Depth 0
↓
Depth 1
↓
Depth 2
↓
Depth 3
↓ ... until the goal is found.

Example Suppose:



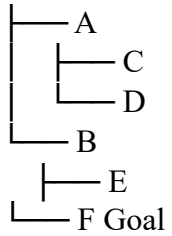
Iteration 1

Depth limit = 0

S

Goal not found.

Iteration 2
Depth limit = 1
S
├── A └──
B
Goal not found.

Iteration 3
Depth limit = 2
S


not found.

Iteration 4
Depth limit = 3
The search explores another level and finds G.
Therefore, the solution is found.

IDS Advantages
1. Low Memory
IDS uses DFS-like memory.
Space complexity is approximately:
\[\boxed{O(bd)} \] where:

- \(b\) = branching factor
- \(d\) = depth of shallowest goal

2. Complete
IDS is complete for finite branching factors.
3. Optimal for Equal Step Costs
When every action has the same cost, IDS finds a shallowest solution.

IDS Disadvantages
1. Repeated Work
The upper levels of the tree are searched repeatedly.
For example:
Depth 1 → Search S
Depth 2 → Search S again
Depth 3 → Search S again

Therefore, some nodes are expanded multiple times.
2. Not Suitable for Different Costs IDS is
primarily depth-based.
If actions have different costs, the shallowest solution may not be the cheapest solution.
3. Can Be Computationally Expensive
Although memory usage is low, repeated searches increase computation.

IDS Complexity
Time complexity: \[
\boxed{O(b^d)} \]
Space complexity:
\[\boxed{O(bd)} \]
where:

- \(b\) = branching factor
- \(d\) = depth of shallowest solution

UCS vs IDS

Feature	UCS	IDS
Search type	Cost-based	Depth-based
Uses heuristic	No	No
Complete	Yes, under standard positive-cost assumptions	Yes for finite branching
Optimal	Yes for non-negative costs	Yes when all step costs are equal
Memory	High	Low
Time	Can be high	Can be high due to repetition
Different costs	Suitable	Not suitable
Repeated expansion	Less repetition	High
Main advantage	Least-cost path	Low memory

Intelligent Agents and Maze Problem
The type of intelligent agent affects how the robot makes decisions.

1. Simple Reflex Agent
A simple reflex robot uses current percepts.
For example:
IF obstacle ahead
THEN turn left
Limitation
It does not remember previously visited locations.
Therefore, it can get stuck in loops:
A → B → A → B → A

2. Model-Based Agent

A model-based agent maintains an internal representation of the maze.
It can remember:

- Visited cells
- Obstacles
- Previous movements
- Possible routes Example:

Sensor information
↓
Update internal map
↓
Select next action
This is more suitable for an unknown maze.

3. Goal-Based Agent
A goal-based agent has an explicit objective:
$$\boxed{\text{Reach the maze exit}}$$
It evaluates possible paths and selects actions that help reach the goal.

4. Utility-Based Agent
A utility-based agent can consider additional factors such as:

- Distance
- Energy
- Safety
- Risk
- Time

For example:
Path A = 10 steps, safe
Path B = 7 steps, risky
A utility-based agent can balance these factors.

Unknown Maze and Agent Design
Because the maze is unknown, the robot should maintain information about what it has already discovered.
Therefore, a **model-based agent** is more appropriate than a purely simple reflex agent. If the robot must also balance:

- Distance
- Energy
- Safety
- Time

then a **utility-based model-based agent** becomes more appropriate.

Recommended Combination
For a basic unknown maze where every movement costs 1:
$$\boxed{\text{Model-Based Agent + IDS}}$$ can be useful when **memory is limited** and the objective is to find a shallow solution.

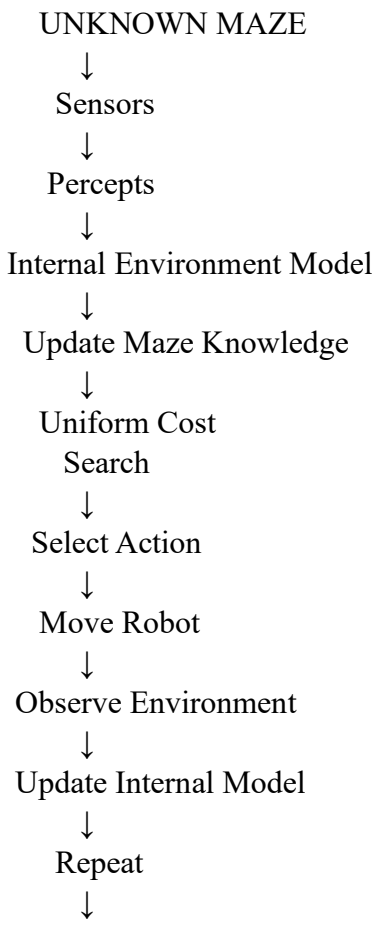
However, if the maze has **different movement costs**, then:
\[\boxed{\text{Model-Based Agent + UCS}} \] is more
appropriate because UCS considers actual path cost.

Why Model-Based + UCS?
For a practical robot, I would use a **model-based agent with UCS**, particularly when path costs can differ.
Reason 1 – Unknown Environment
The model-based agent can store discovered information.
Reason 2 – Memory It
can remember:

- Visited locations
- Obstacles
- Routes
- Dead ends **Reason 3 – Cost** UCS considers:

\[g(n)=\text{actual path cost} \]
Reason 4 – Optimality
With non-negative action costs, UCS can find a least-cost solution.
Reason 5 – Decision Making
The agent can update its internal model and search for a new path when new information becomes available.

Overall Architecture



Reach Goal

Final Conclusion

The unknown-maze problem can be solved using uninformed search algorithms such as **UCS and IDS**. UCS selects the node with the lowest cumulative path cost and is particularly suitable when action costs differ. IDS repeatedly performs depth-limited search and requires much less memory, making it useful when all actions have equal cost and memory is constrained.

The robot should maintain an internal representation of the environment because the maze is initially unknown. Therefore, a **model-based agent** is more suitable than a simple reflex agent. When all movements have equal cost and memory is the main concern, **IDS** is useful; when path costs vary and finding a least-cost route is important, **UCS with a model-based agent** is a suitable combination.